

Infosec

Leonardo González

Seguridad: De POC a Producción - El Viaje a la Resiliencia



Apertura: Escala e Importancia

La realidad global de las ciberamenazas y el cambio de contexto de riesgo



Mapa de Amenazas Globales en Tiempo Real

Aumento de Superficie Atacable



COSTO ESPERADO DEL RIESGO SUBE SIGNIFICATIVAMENTE
(Incluso sin cambios de código)

Aumento del Valor del Objetivo

Aplicaciones críticas

POC
(Entorno Controlado)

PRODUCCIÓN
(Escenario Real)

La Importancia del Contexto y el Valor del Negocio

Mapa de Superficies de Riesgo

Índice Operativo: Checklist, Backlog y Reporte

INFRAESTRUCTURA Y PLATAFORMA



Nube, red, OS, contenedores, runtime

EDGE Y EXPOSICIÓN



DNS, CDN, WAF, reverse proxy, API gateway

SERVICIOS Y APLICACIÓN



Código, configuración, APIs, feature flags

IDENTIDAD Y ACCESO



Humanos, servicios, agentes, llaves, IAM, RBAC

DATOS



DBs, objetos, colas, logs, RAG, retención

SISTEMA DE IA



Modelo e inferencia, prompts, contexto, agentes y herramientas, evals y guardrails

SDLC Y SUPPLY CHAIN



Dependencias, CI/CD, IaC, imágenes, firmas

OBSERVABILIDAD Y RESPUESTA



Auditoría, detección, IR

Heurística de Priorización (POC a Producción)



ENFOQUE EN IMPACTO IRREVERSIBLE

↓ Exfiltración ↓ Ejecución Remota ↓ Escalamiento de Privilegios ↓ Destrucción de Datos

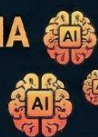
EMPEZAR
AQUÍ



PRIORIDAD: AMPLIFICADO CON IA



Agencia, Herramientas, Contexto y Supply Chain



Frameworks y taxonomías: OWASP y Seguridad Avanzada

1) OWASP Top 10 Web Application Security Risks 2025

Como mapa base para aplicaciones y APIs

-  **A01:2025** Broken Access Control
-  **A02:2025** Security Misconfiguration
-  **A03:2025** Software Supply Chain Failures
-  **A04:2025** Cryptographic Failures
-  **A05:2025** Injection
-  **A06:2025** Insecure Design
-  **A07:2025** Authentication Failures
-  **A08:2025** Software or Data Integrity Failures
-  **A09:2025** Security Logging and Alerting Failures
-  **A10:2025** Mishandling of Exceptional Conditions

2) OWASP Top 10 for LLM Applications 2025

Taxonomía específica de riesgos en apps con LLM

-  **LLM01:2025** Prompt Injection
-  **LLM02:2025** Sensitive Information Disclosure
-  **LLM03:2025** Supply Chain
-  **LLM04:2025** Data and Model Poisoning
-  **LLM05:2025** Improper Output Handling
-  **LLM06:2025** Excessive Agency
-  **LLM07:2025** System Prompt Leakage
-  **LLM08:2025** Vector and Embedding Weaknesses
-  **LLM09:2025** Misinformation
-  **LLM10:2025** Unbounded Consumption

Lectura práctica para ingeniería: estos riesgos suelen colapsar en **4 controles estructurales**

-  Separación estricta entre instrucciones y datos (y etiquetado de contenido no confiable)
-  Validación y "gating" determinístico de outputs antes de ejecutar acciones
-  Privilegios mínimos y "least agency" para herramientas y side effects
-  Presupuestos y límites (tokens, costo, rate, tiempo) para evitar consumo no acotado

3) OWASP Top 10 for Agentic Applications 2026

Para sistemas con agentes que planifican y actúan con herramientas

-  **ASI01** Agent Goal Hijack
-  **ASI02** Tool Misuse and Exploitation
-  **ASI03** Identity and Privilege Abuse
-  **ASI04** Agentic Supply Chain Vulnerabilities
-  **ASI05** Unexpected Code Execution (RCE)
-  **ASI06** Memory & Context Poisoning
-  **ASI07** Insecure Inter-Agent Communication
-  **ASI08** Cascading Failures
-  **ASI09** Human-Agent Trust Exploitation
-  **ASI10** Rogue Agents

Traducción inmediata a hardening del arnés

-  Permisos de herramientas
-  Identidad de agentes
-  Canales inter-agente
-  Memoria/contexto persistente

Controles: de POC a producción



A. Prevención (qué endurecer primero)

POC Producción



INFRA Y PLATAFORMA

Baselines de hardening del runtime: imágenes mínimas, usuarios no root, capacidades mínimas, aislamiento, parches y actualización; IaC con defaults seguros y revisión automatizada



SERVICIOS Y APLICACIÓN

Autorización como política (A01) y no no como condicionales dispersos; Validación de input, encoding, contra inyecciones (A05)



DATOS

Clasificación simple (PII, secretos, internos, públicos), retención, cifrado en tránsito y reposo, y auditoría de acceso
Para RAG: higiene de fuentes y límites de acceso, output handling determinístico (LLM05), y políticas contra prompt injection (LLM01)



SUPPLY CHAIN



- Gestión de dependencias (SCA), bloqueo por políticas, SBOM, firma y verificación cuando aplique
- Controles para "agentic supply chain": prompts, skills, plugins, MCP servers, tool adapters



EDGE Y EXPOSICIÓN

TLS bien configurado, headers, rate limiting, WAF donde aplique, y logs de borde; Política de CORS y endpoints públicos explícitos



IDENTIDAD Y ACCESO

MFA, rotación, separación de roles, Para agente: agente y por herramienta, scopes mínimos y expiración corta



B. Detección y respuesta (cuando algo falla)



LOGS ÚTILES

Log file with identity trace, request ID, decisions, herramientas, changes



ALERTAS ACCIONABLES

Con severidad y unbooks



IR BÁSICO

Triage, contención, erradicación, recuperación, postmortem



C. Continuidad y resiliencia

Human-Agent BIA exploitation

A110 Rogue Agents



BIA PRIORITARIA

Para priorizar sistemas críticos y dependencias



DRP CON OBJETIVOS

RPO, RTO, WRT, MTD



INMUTABILIDAD DE EVIDENCIA

Inmutabilidad y aislamiento de logs y evidencia cuando aplique



EJERCICIOS Y COMUNICACIÓN

Ejercicios de recuperación y canales de comunicación alternos



IA aplicada a seguridad



1) "Security copilots" para hardening, sin crear nuevos riesgos

patrón recomendado: agentes para acelerar análisis y ejecución repetible, con guardrails (REDUCIR BLAST RADIUS)

Guardrails mínimos del arnés:



Allowlist explícita de herramientas y comandos, contra inyecciones



Separación de credenciales por entorno (dev, staging, prod) y por rol.



Aprobación humana para side effects altos (borrar datos, rotar llaves, cambiar IAM, desplegar).



Logging estructurado del "agentic loop": objetivo, plan, comandos, diffs, outputs, verificación.



2) SAST con agentes, guiado por OWASP

Flujo típico:



1. Generar checklist de categorías OWASP aplicables (web, API, auth, crypto, supply chain).



2. Ejecutar SAST (Semgrep o CodeQL) y scanning de secretos.



3. Triage agencial: agrupar hallazgos por riesgo, impacto, explotabilidad.



4. Proponer fixes con pruebas de regresión. y dependencias



5. Generar "Security Backlog" con criterios de aceptación y comandos de verificación.



3) Pentesting asistido por IA (mención breve y responsable)

Shannon (KeygraphHQ) – es un pentester autónomo que busca vectores y ejecuta exploits reales para probar explotabilidad en apps web.

Uso pedagógico en esta clase: entender cómo cambia el juego cuando el atacante también automatiza exploración y explotación. Por qué eso hace valiosos los controles de autorización, segmentación, límites, y observabilidad.

Apex vs Shannon Lite

Arnés amplio de pentesting autónomo vs especialista en AppSec web con conocimiento del código fuente



Posicionamiento: Herramienta general de pentesting autónomo

- Soporta pentesting de caja negra y caja blanca
- Diseñada para flujos de trabajo terminal-first
- Incluye uso headless / CI-CD
- Tiene un modo operador para investigación manual y encadenamiento de exploits
- Soporta memoria persistente entre evaluaciones
- Ofrece un contenedor Kali opcional con herramientas de pentesting
- Licencia: Apache-2.0

Más amplio y más flexible

Idea central: Se solapan, pero no son el mismo producto.

- **Apex** = herramienta más amplia de pentesting autónomo con enfoque tanto de caja negra como de caja blanca
- **Shannon Lite** = pentester más enfocado y de caja blanca para aplicaciones web y APIs, con énfasis en validación mediante explotación

Diferencia principal

Más amplio	Alcance	Más enfocado
Caja negra + caja blanca	Estilo de prueba	Solo caja blanca
Pentesting autónomo general	Enfoque	AppSec web con conocimiento del código fuente
Orientado a terminal / operador	Flujo de trabajo	Orientado a prueba mediante explotación
Apache-2.0	Licencia	AGPL-3.0

Ajuste práctico

Elige **Apex** si quieres:

> Flexible, flujos
> amplios, terminal,
terminal, general ofensivo

Elige **Shannon Lite** si quieres:

</> Código fuente web,
validación fuerte, AppSec
especializado, flujo definido

Conclusión:

- **Shannon Lite** parece más maduro para pentesting de aplicaciones web con conocimiento del código fuente.
- **Apex** parece más amplio como arnés general de pentesting autónomo.

Shannon Lite

Posicionamiento: Pentester autónomo con IA de caja blanca para aplicaciones web y APIs

- Explícitamente solo caja blanca
- Enfocado en aplicaciones web y APIs
- Usa análisis con conocimiento del código fuente
- Enfatiza la prueba mediante explotación
- Los hallazgos se validan con pruebas de concepto funcionales
- Afirmación pública de benchmark: 96.15% (100/104 exploits) en una variante sin pistas y con conocimiento del código fuente
- Licencia: AGPL-3.0

Más enfocado y más especializado

MENSAJES FINALES PARA FIJAR LA SESIÓN

HARDENING EN PRODUCCIÓN: UNA COMBINACIÓN FUNDAMENTAL



EL PERÍMETRO EXPANDIDO DE LLMs Y AGENTES



LA META PRÁCTICA: DE INTENCIONES A ACCIONES



La meta práctica de este día es convertir riesgos en tareas verificables y dejar evidencia, no solo intenciones.